FEU Institute of Technology
COLLEGE OF ENGINEERING • COLLEGE OF COMPUTER STUDIES

Computer System and Architecture

Machine Problem

FLAB4

Charles Gabriel Marcelo	Doc. Hadji Tejuco
Name of Student	Name of Professor
November 5, 2026	November 5, 2026
Data Performed	Date Submitted

List the steps that you perform to come up with the program.

Paste the source code and explain the functionalities and take a screenshot of the final output.

Program Description

The Georgette Eatery program is a NASM 32-bit assembly application that uses procedures and macros to manage a simple food ordering system. It displays six food options with their prices, validates the user's selection, and records each ordered item and its price in receipt.txt. After each order, the user enters Y to order again or N to finish. When ordering ends, the program displays "Thank you! Please come again!", closes the receipt file, and reopens it to display all recorded orders.

COMPILATION PROCESS:

```
@kizern ➜/workspaces/bananabread/FLAB4 (main) $ nasm -f elf32 georgette_eatery.asm -o georgette_eatery.o
@kizern ➜/workspaces/bananabread/FLAB4 (main) $ ld -m elf_i386 georgette_eatery.o -o georgette_eatery
@kizern ➜/workspaces/bananabread/FLAB4 (main) $ ./georgette_eatery

========== GEORGETTE EATERY ==========
A. Rice : Plain Rice        Php 5.00
B. Rice : Java Rice         Php 10.00
C. Ulam 1: Fish             Php 35.00
D. Ulam 2: Pork             Php 50.00
```

PROGRAM OUTPUT:

```
========== GEORGETTE EATERY ==========
A. Rice : Plain Rice        Php 5.00
B. Rice : Java Rice         Php 10.00
C. Ulam 1: Fish             Php 35.00
D. Ulam 2: Pork             Php 50.00
E. Ulam 3: Vegetables       Php 20.00
F. Dessert: Saging con yelo Php 60.00
Enter your choice [A-F]: A
You ordered: Rice : Plain Rice        Php 5.00
Do you want to order again? [Y/N]: Y

========== GEORGETTE EATERY ==========
A. Rice : Plain Rice        Php 5.00
B. Rice : Java Rice         Php 10.00
C. Ulam 1: Fish             Php 35.00
D. Ulam 2: Pork             Php 50.00
E. Ulam 3: Vegetables       Php 20.00
F. Dessert: Saging con yelo Php 60.00
Enter your choice [A-F]: c
You ordered: Ulam 1: Fish             Php 35.00
Do you want to order again? [Y/N]: y

========== GEORGETTE EATERY ==========
A. Rice : Plain Rice        Php 5.00
B. Rice : Java Rice         Php 10.00
C. Ulam 1: Fish             Php 35.00
D. Ulam 2: Pork             Php 50.00
E. Ulam 3: Vegetables       Php 20.00
F. Dessert: Saging con yelo Php 60.00
```

```
Enter your choice [A-F]: e
You ordered: Ulam 3: Vegetables       Php 20.00
Do you want to order again? [Y/N]: y

========== GEORGETTE EATERY ==========
A. Rice : Plain Rice        Php 5.00
B. Rice : Java Rice         Php 10.00
C. Ulam 1: Fish             Php 35.00
D. Ulam 2: Pork             Php 50.00
E. Ulam 3: Vegetables       Php 20.00
F. Dessert: Saging con yelo Php 60.00
Enter your choice [A-F]: f
You ordered: Dessert: Saging con yelo Php 60.00
Do you want to order again? [Y/N]: n

Thank you! Please come again!

Contents of receipt.txt:
========== GEORGETTE EATERY ==========
RECEIPT
-------------------------------------
Rice : Plain Rice        Php 5.00
Ulam 1: Fish             Php 35.00
Ulam 3: Vegetables       Php 20.00
Dessert: Saging con yelo Php 60.00
-------------------------------------
Thank you! Please come again!
@kizern ➜/workspaces/bananabread/FLAB4 (main) $
```

TEST CASES:

#1 (Invalid Menu Choice):

```
@kizern ➜/workspaces/bananabread/FLAB4 (main) $ ./georgette_eatery

========== GEORGETTE EATERY ==========
A. Rice : Plain Rice        Php 5.00
B. Rice : Java Rice         Php 10.00
C. Ulam 1: Fish             Php 35.00
D. Ulam 2: Pork             Php 50.00
E. Ulam 3: Vegetables       Php 20.00
F. Dessert: Saging con yelo Php 60.00
Enter your choice [A-F]: ffsdfsddf
Invalid choice. Enter one letter from A to F.
Enter your choice [A-F]: 23244
Invalid choice. Enter one letter from A to F.
Enter your choice [A-F]: &%^*$
Invalid choice. Enter one letter from A to F.
Enter your choice [A-F]:
```

#2 (Invalid "Order Again?" Input):

```
Enter your choice [A-F]: A
You ordered: Rice : Plain Rice        Php 5.00
Do you want to order again? [Y/N]: 1213
Invalid answer. Enter Y or N.
Do you want to order again? [Y/N]: &&$#*$&$
Invalid answer. Enter Y or N.
Do you want to order again? [Y/N]: FJJFSDJFSNJSF
Invalid answer. Enter Y or N.
Do you want to order again? [Y/N]:
```

#3 (Receipt.txt & Console Output Comparison):

```
Do you want to order again? [Y/N]: N

Thank you! Please come again!

Contents of receipt.txt:
========== GEORGETTE EATERY ==========
RECEIPT
--------------------------------------
Rice : Java Rice          Php 10.00
Ulam 1: Fish              Php 35.00
Dessert: Saging con yelo  Php 60.00
Ulam 3: Vegetables        Php 20.00
--------------------------------------
Thank you! Please come again!
```

georgette_eatery.asm U | receipt.txt U

FLAB4 > receipt.txt ...abread/FLAB4/georgette_eatery.asm • Untracked

```
1   ========== GEORGETTE EATERY ==========
2   RECEIPT
3   --------------------------------------
4   Rice : Java Rice          Php 10.00
5   Ulam 1: Fish              Php 35.00
6   Dessert: Saging con yelo  Php 60.00
7   Ulam 3: Vegetables        Php 20.00
8   --------------------------------------
9   Thank you! Please come again!
```

SOURCE CODE:

```
global _start

%define SYS_EXIT   1
```

```
%define SYS_READ   3
%define SYS_WRITE  4
%define SYS_OPEN   5
%define SYS_CLOSE  6
%define STDIN      0
%define STDOUT     1
%define STDERR     2
%define EINTR      4
%define O_RDONLY   0
%define CREATE_TRUNC_WRITE 577 ; O_WRONLY | O_CREAT | O_TRUNC

%macro PRINT 1
    mov ebx, STDOUT
    mov ecx, %1
    mov edx, %1 %+ _len
    call write_all
    test eax, eax
    js io_error
%endmacro

%macro SAVE 1
    mov ebx, [receipt_fd]
    mov ecx, %1
    mov edx, %1 %+ _len
    call write_all
    test eax, eax
    js io_error
%endmacro

SECTION .data
    filename db 'receipt.txt', 0
    receipt_fd dd -1

    menu db 10, '========== GEORGETTE EATERY ==========', 10
        db 'A. Rice : Plain Rice        Php 5.00', 10
        db 'B. Rice : Java Rice         Php 10.00', 10
        db 'C. Ulam 1: Fish             Php 35.00', 10
        db 'D. Ulam 2: Pork             Php 50.00', 10
        db 'E. Ulam 3: Vegetables       Php 20.00', 10
        db 'F. Dessert: Saging con yelo Php 60.00', 10
    menu_len equ $ - menu

    choice_prompt db 'Enter your choice [A-F]: '
```

```
    choice_prompt_len equ $ - choice_prompt
    again_prompt db 'Do you want to order again? [Y/N]: '
    again_prompt_len equ $ - again_prompt
    invalid_choice db 'Invalid choice. Enter one letter from A to F.', 10
    invalid_choice_len equ $ - invalid_choice
    invalid_again db 'Invalid answer. Enter Y or N.', 10
    invalid_again_len equ $ - invalid_again
    ordered db 'You ordered: '
    ordered_len equ $ - ordered
    goodbye db 10, 'Thank you! Please come again!', 10
    goodbye_len equ $ - goodbye

    receipt_header db '========== GEORGETTE EATERY ==========', 10
                   db 'RECEIPT', 10
                   db '-------------------------------------', 10
    receipt_header_len equ $ - receipt_header
    receipt_footer db '-------------------------------------', 10
                   db 'Thank you! Please come again!', 10
    receipt_footer_len equ $ - receipt_footer
    receipt_title db 10, 'Contents of receipt.txt:', 10
    receipt_title_len equ $ - receipt_title
    error_message db 'Error: unable to read/write input, output, or receipt.txt.', 10
    error_message_len equ $ - error_message

    item_a db 'Rice : Plain Rice        Php 5.00', 10
    item_a_len equ $ - item_a
    item_b db 'Rice : Java Rice         Php 10.00', 10
    item_b_len equ $ - item_b
    item_c db 'Ulam 1: Fish             Php 35.00', 10
    item_c_len equ $ - item_c
    item_d db 'Ulam 2: Pork             Php 50.00', 10
    item_d_len equ $ - item_d
    item_e db 'Ulam 3: Vegetables       Php 20.00', 10
    item_e_len equ $ - item_e
    item_f db 'Dessert: Saging con yelo Php 60.00', 10
    item_f_len equ $ - item_f

    items dd item_a, item_a_len, item_b, item_b_len
          dd item_c, item_c_len, item_d, item_d_len
          dd item_e, item_e_len, item_f, item_f_len

SECTION .bss
    input_byte resb 1
```

```
    receipt_buffer resb 512

SECTION .text
_start:
    mov eax, SYS_OPEN
    mov ebx, filename
    mov ecx, CREATE_TRUNC_WRITE
    mov edx, 0o644
    int 0x80
    test eax, eax
    js io_error
    mov [receipt_fd], eax
    SAVE receipt_header

.menu_loop:
    call display_menu
.choose:
    PRINT choice_prompt
    call read_choice
    cmp eax, -1
    je .finish
    cmp eax, 'A'
    jb .bad_choice
    cmp eax, 'F'
    ja .bad_choice

    sub eax, 'A'
    mov esi, [items + eax*8]
    mov edi, [items + eax*8 + 4]
    call record_order

.ask_again:
    PRINT again_prompt
    call read_choice
    cmp eax, -1
    je .finish
    cmp eax, 'Y'
    je .menu_loop
    cmp eax, 'N'
    je .finish
    PRINT invalid_again
    jmp .ask_again
```

```
.bad_choice:
    PRINT invalid_choice
    jmp .choose

.finish:
    SAVE receipt_footer
    PRINT goodbye
    call close_receipt
    call display_receipt
    mov eax, SYS_EXIT
    xor ebx, ebx
    int 0x80

display_menu:
    PRINT menu
    ret

record_order:
    mov ebx, [receipt_fd]
    mov ecx, esi
    mov edx, edi
    call write_all
    test eax, eax
    js io_error
    PRINT ordered
    mov ebx, STDOUT
    mov ecx, esi
    mov edx, edi
    call write_all
    test eax, eax
    js io_error
    ret

read_choice:
    push ebx
    push ecx
    push edx
    push esi
    push edi
    xor esi, esi
    xor edi, edi
.read:
    mov eax, SYS_READ
```

```
    mov ebx, STDIN
    mov ecx, input_byte
    mov edx, 1
    int 0x80
    cmp eax, -EINTR
    je .read
    test eax, eax
    js io_error
    jz .eof
    movzx eax, byte [input_byte]
    cmp al, 10
    je .line_done
    cmp al, 13
    je .read
    test esi, esi
    jnz .extra_character
    mov edi, eax
.extra_character:
    cmp esi, 2
    jae .read
    inc esi
    jmp .read
.eof:
    mov eax, -1
    test esi, esi
    jz .return
.line_done:
    xor eax, eax
    cmp esi, 1
    jne .return
    mov eax, edi
    cmp al, 'a'
    jb .return
    cmp al, 'z'
    ja .return
    sub eax, 'a' - 'A'
.return:
    pop edi
    pop esi
    pop edx
    pop ecx
    pop ebx
    ret
```

```
write_all:
    test edx, edx
    jz .done
.write:
    mov eax, SYS_WRITE
    int 0x80
    cmp eax, -EINTR
    je .write
    test eax, eax
    js .return
    jz .zero_write
    add ecx, eax
    sub edx, eax
    jnz .write
.done:
    xor eax, eax
.return:
    ret
.zero_write:
    mov eax, -5
    ret

close_receipt:
    mov eax, SYS_CLOSE
    mov ebx, [receipt_fd]
    mov dword [receipt_fd], -1
    int 0x80
    test eax, eax
    js io_error
    ret

display_receipt:
    PRINT receipt_title
    mov eax, SYS_OPEN
    mov ebx, filename
    mov ecx, O_RDONLY
    xor edx, edx
    int 0x80
    test eax, eax
    js io_error
    mov [receipt_fd], eax
.read:
```

```
    mov eax, SYS_READ
    mov ebx, [receipt_fd]
    mov ecx, receipt_buffer
    mov edx, 512
    int 0x80
    cmp eax, -EINTR
    je .read
    test eax, eax
    js io_error
    jz .done
    mov edx, eax
    mov ebx, STDOUT
    call write_all
    test eax, eax
    js io_error
    jmp .read
.done:
    call close_receipt
    ret

io_error:
    mov ebx, STDERR
    mov ecx, error_message
    mov edx, error_message_len
    call write_all
    mov ebx, [receipt_fd]
    cmp ebx, 0
    jl .exit
    mov eax, SYS_CLOSE
    int 0x80
.exit:
    mov eax, SYS_EXIT
    mov ebx, 1
    int 0x80

SECTION .note.GNU-stack noalloc noexec nowrite progbits
```